

En esta evidencia se presenta la implantación del sistema de mi equipo AMP. La finalidad de esta evidencia o etapa es comprobar que el proyecto pudiera funcionar correctamente fuera del desarrollo inicial, conectados con la base de datos y permitiendo el acceso de los usuarios mediante inicio de sesión.

El sistema fue trabajado como una aplicación web utilizando Node.js , Express, EJS y PostgreSQL. Durante la implantación se revisó que las principales funciones del sistema estuvieran disponibles, como el login, la navegación entre vistas, el control de roles y la conexión con la base de datos.

El objetivo de la implantación fue poner en funcionamiento el sistema para ver que funcione correctamente y en caso que reporte un fallo o bug corregirlo para que funcione correctamente. También se buscó que la autenticación de usuarios, las sesiones y el acceso por rol funcionaran correctamente.

Para implementar el sistema se utilizaron herramientas de desarrollo web tanto del lado del cliente como del servidor. Para backend se trabajó con [Node.js](#) y Express mientras que las vistas fueron desarrolladas con EJS, HTML, CSS y JS.

La base de datos utilizada fue PostgreSQL, la cual permite almacenar la información del sistema. Además, se usaron sesiones para mantener activo el usuario después de iniciar sesión, así como configuraciones de seguridad para proteger la aplicación.

Procedimiento de implantación:

Primero se revisó que el proyecto tuviera instaladas todas las dependencias necesarias incluyendo para hacer las pruebas. Para esto se utilizó npm pero por cosas fuera de nuestras manos se decidió cambiar a pnpm, ambas se utilizaron para verificar que los paquetes requeridos estuvieran configurados correctamente y además como buena práctica.

Después se configuró la conexión con la base de datos mediante las variables de entorno correspondientes. Esto permitió que el sistema pudiera consultar y guardar información sin colocar datos importantes dentro del código.

Posteriormente se revisó el funcionamiento del inicio de sesión y el manejo de sesiones. Nos dimos cuenta que en local funciona todo correctamente pero al hacer el despliegue con Vercel nos dimos cuenta gracias al Socio Formador que al momento de ingresar a la dashboard (después de hacer login) si interactuaba con Vista General se crasheaba y mandaba un error 500 o simplemente cerraba la sesión, cosa que ya lo resolvimos.

Problemas encontrados:

Como lo comente anteriormente, tuvimos ese problemita con la dashboard pero otro problema que tuvimos fue con las cookies porque al momento de hacer login nos marcaba error y además los botones de “Aceptar me gustan las galletitas” y “No me gustan las galletitas” no tenían funcionalidad, si aparecían pero al momento de dar click no se reportaba la acción.

Mucho antes de esos problemas tuvimos un problemita con login y sign in ya que cada apartado chocaba entre sí y además no se registraba en la base de datos y mandaba error. Actualmente ya están corregidos estos errores.

Resultados

Con la implantación se logró comprobar que el sistema AMP funciona correctamente en sus módulos principales. El sistema permite iniciar sesión, navegar entre vistas, validar roles y conectarse con la base de datos.

3. Manual de usuario

Este manual tiene como objetivo explicar el funcionamiento básico del sistema AMP para permitir que cualquier usuario pueda utilizar las funciones principales.

Requisitos:

- Navegador actualizado
- Conexión a internet
- Usuario y contraseña

Inicio de sesión

1. Abrir el sistema
2. Ingresar usuario y contraseña
3. Dar clic o enter
4. Esperar redirección al Dashboard

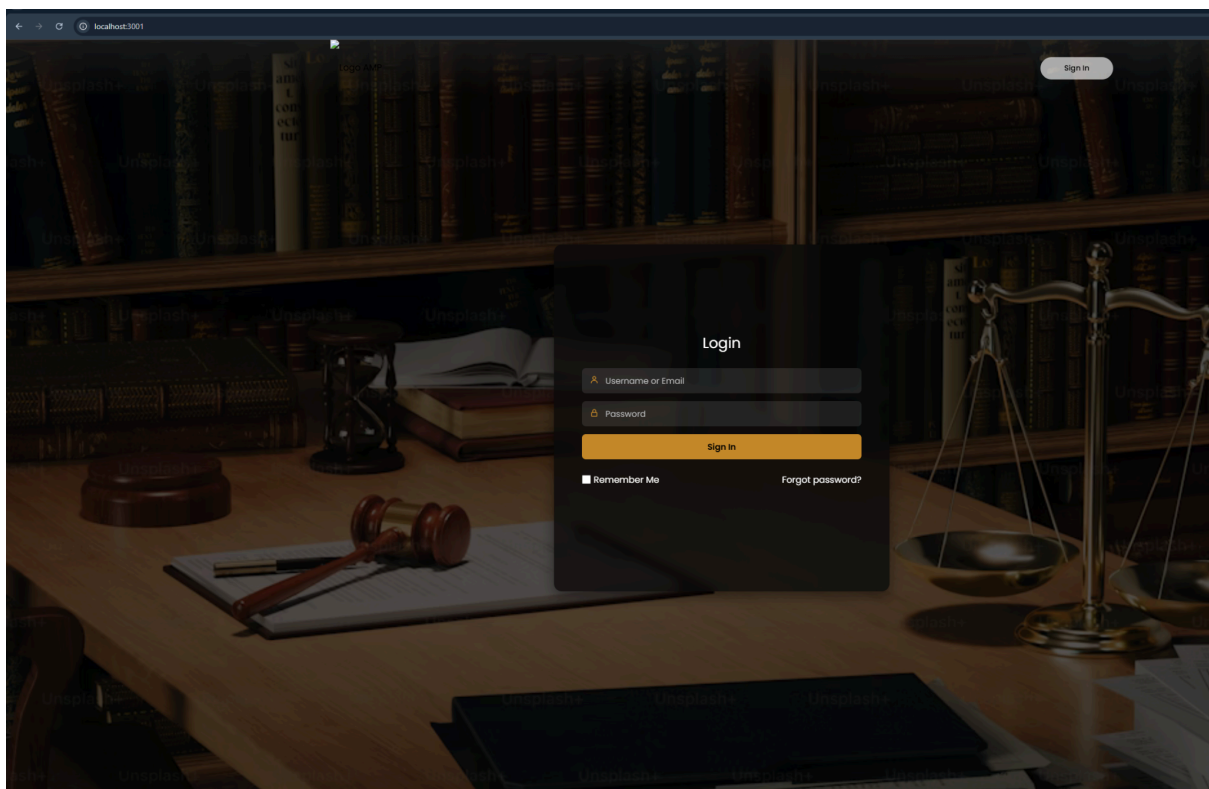


Figura 1. Visualización del login

Navegación del sistema

Desde el menu lateral el usuario podra acceder a:

- Dashboard
- Buzón
- Reportes
- Historial
- Configuración

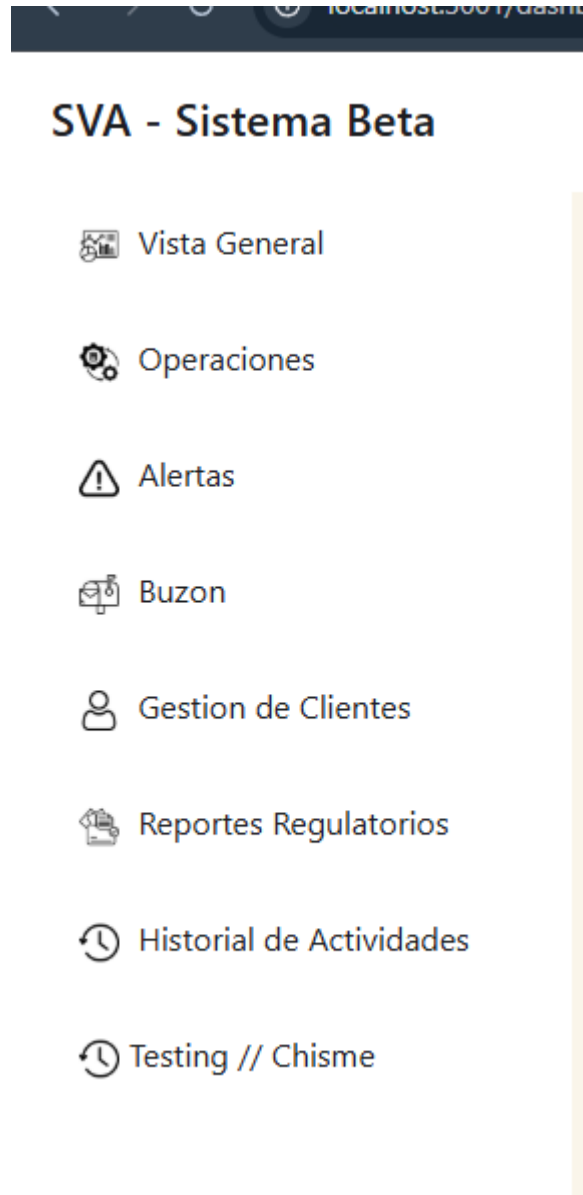


Figura 2. Visualización de la side-bar

- Cierre de sesión
- Seleccionar cerrar sesión
 - Confirmar salida
 - El sistema regresará al inicio

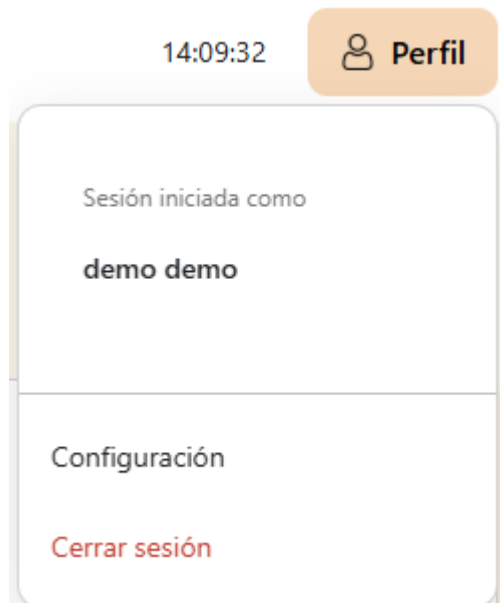


Figura 3. Visualización de cerrar sesión

4. Manual de instalación

Requisitos previos

- [Node.js](#) instalado
- PostgreSQL configurado
- npm instalado

Instalación

1. Descargar el repositorio
2. Abrir terminal
3. Ejecutar
npm i (npm install)
4. Configurar archivo .env
5. Ejecutar node [app.js](#)
6. Abrir navegador (<http://localhost:3001>)

Validación

- Inicio de sesión
- Conexión con BD
- Navegación correcta

```
PS C:\Users\David\Downloads\TEST935PM29MAY\TEST> node app.js
◇ injected env (3) from .env // tip: # multiple files { path: ['.env.local', '.env'] }
◇ injected env (0) from .env // tip: # override existing { override: true }
http://localhost:3001
SÍ ENTRÓ AL LOGIN
BODY: {
  _csrf: '04d5092fe4a09a805f5df27c242120fbdc4bf58d4ac2d3ad61778a472872aab3.4415ffe41b2a3335305c40a47218bd815a1390f671aba4c0b078bca8c7f29561',
  username: 'demo@demo.com',
  password: 'demo'
}
CORREO A BUSCAR: demo@demo.com
USUARIO: {
  ID_Usuario: 6,
  Nombre: 'demo',
  Apellido: 'demo',
  Correo: 'demo@demo.com',
  Contraseña: '$2b$12$soHTuwo/F/Zvcrk1q3uRGepIGsJUVXeYbDATXabTdZnLJBxRRFIbu'
}
PASSWORD: true
```

Figura 4. Visualización del funcionamiento de la aplicación

En general esta etapa fue importante porque permitió identificar errores de configuración y corregirlos antes de hacer la entrega final. También ayudó a reforzar conocimientos sobre despliegue, sesiones, bases de datos y el funcionamiento de una aplicación web

```
test/usuarios.test.js
• Console

console.log
  ◇ injected env (3) from .env // tip: # custom filepath { path: '/custom/path/.env' }
  at _log (node_modules/dotenv/lib/main.js:131:11)

console.log
  Obtener Usuarios
  at Object.log [as ObtenerUsuarios] (models/usuarios.model.js:5:5)

console.log
  Obtener Usuarios
  at Object.log [as ObtenerUsuariosActivos] (models/usuarios.model.js:38:5)

FAIL test/dashboard.test.js
• Console

console.log
  ◇ injected env (3) from .env // tip: ~ auth for agents [www.vestauth.com]
  at _log (node_modules/dotenv/lib/main.js:131:11)

• Dashboard Controller > renderiza dashboard si hay usuario

expect(jest.fn()).toHaveBeenCalled()

Expected number of calls: >= 1
Received number of calls: 0

25 |     });
    |     ^
26 |     dashboardController.renderDashboard(req, res);
    |     expect(res.render).toHaveBeenCalled();
27 |   });
    |   ^
28 |   });
    |   ^
29 | });
    |   ^

at Object.toHaveBeenCalled (test/dashboard.test.js:27:28)
```

Figura 5. Visualización de un error antes de ser corregido

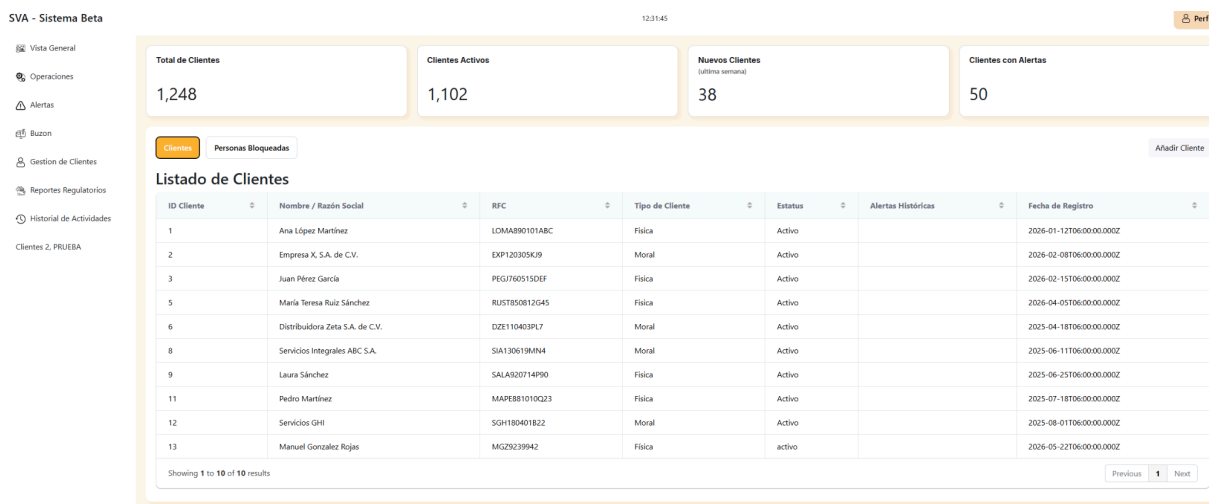


Figura 6. Visualización del dashboard

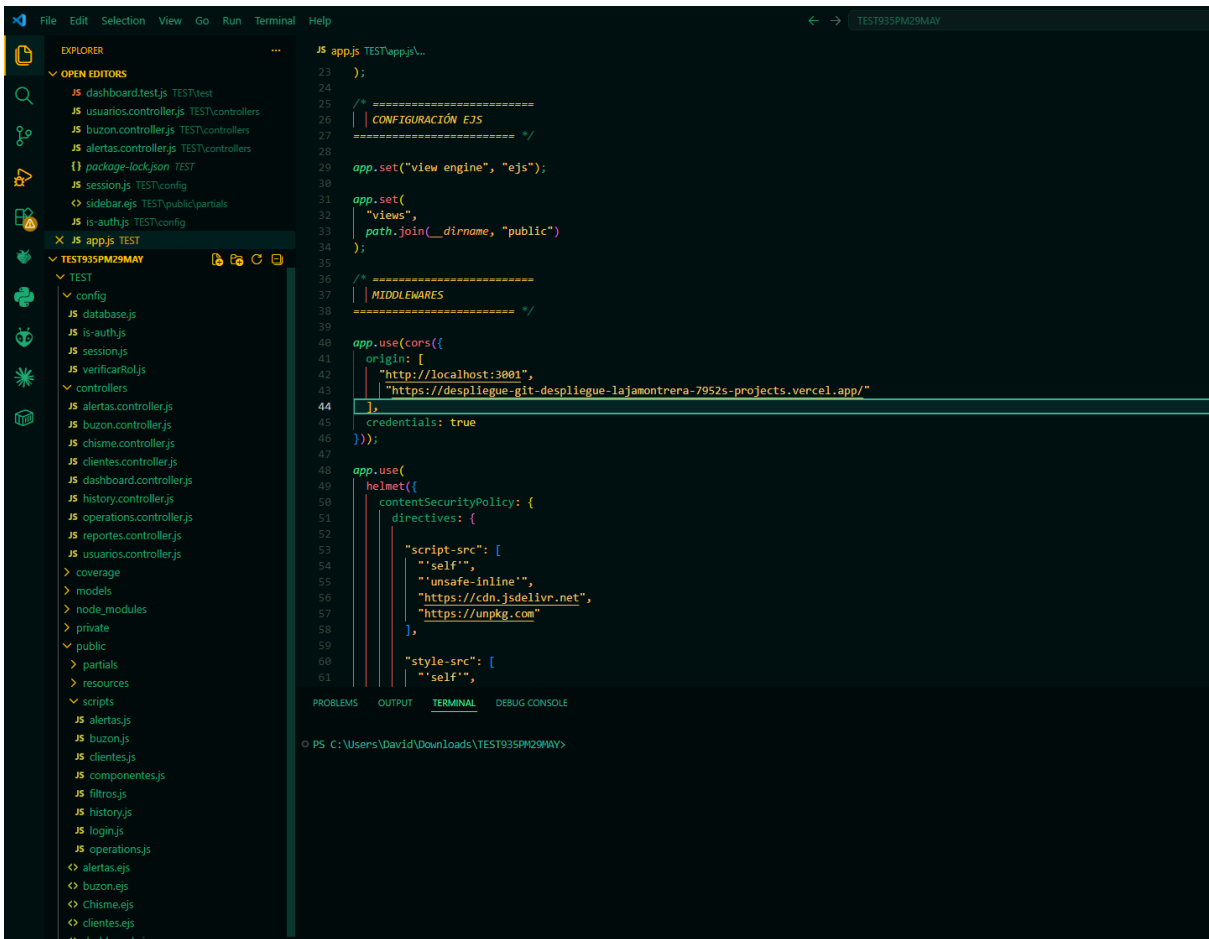


Figura 7. Visualización de [app.js](#)